

Pythonic, and they're a sign that you should use comparison dunder methods.

Summary

Python implements object-oriented features differently than other OOP languages, such as Java or C++. Instead of explicit getter and setter methods, Python has properties that allow you to validate attributes or make attributes read-only.

Python also lets you overload its operators via its dunder methods, which begin and end with double underscore characters. We overload common mathematical operators using the numeric and reflected numeric dunder methods. These methods provide a way for Python's built-in operators to work with objects of the classes you create. If they're unable to handle the data type of the object on the other side of the operator, they'll return the built-in `NotImplemented` value. These dunder methods create and return new objects, whereas the in-place dunder methods (which overload the augmented assignment operators) modify the object in-place. The comparison dunder methods not only implement the six Python comparison operators for objects, but also allow Python's `sort()` function to sort objects of your classes. You might want to use the `eq()`, `ne()`, `lt()`, `le()`, `gt()`, and `ge()` functions in the *operator* module to help you implement these dunder methods.

Properties and dunder methods allow you to write classes that are consistent and readable. They let you avoid much of the boilerplate code that other languages, such as Java, require you to write. To learn more about writing Pythonic code, two PyCon talks by Raymond Hettinger expand on these ideas: “Transforming Code into Beautiful, Idiomatic Python” at <https://youtu.be/OSGv2VnC0go/> and “Beyond PEP 8—Best Practices for Beautiful, Intelligible Code” at <https://youtu.be/wf-BqAjZb8M/> cover some of the concepts in this chapter and beyond.

There's much more to learn about how to use Python effectively. The books *Fluent Python* (O'Reilly Media, 2021) by Luciano Ramalho and *Effective Python* (Addison-Wesley Professional, 2019) by Brett Slatkin provide more in-depth information about Python's syntax and best